

Project Design Phase-II
Solution Requirements (Functional & Non-functional)

Date	30 September 2026
Team ID	SWTID-2026-9682
Project Name	AI StudyBuddy – AI-Augmented Backend Development
Maximum Marks	4 Marks

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User Registration	Registration through form (name, email, password) Duplicate email check Password encrypted with bcryptjs before storage
FR-2	User Authentication	Login with email and password JWT token issued on successful login Token verified on every protected request
FR-3	Study Material Management	Upload or enter study material (title, subject, content) View, update and delete saved materials
FR-4	AI Summary Generation	Generate a concise summary of a material using the Google Gemini API Save the summary against the material
FR-5	AI Flashcard Generation	Generate question–answer flashcards from a material Save and view flashcards
FR-6	AI Quiz Generation	Generate a multiple-choice quiz from a material Save the quiz for later retake
FR-7	AI Study Plan Generation	Enter exam date and available study time Generate and save a personalised study plan
FR-8	Learning Resource History	Dashboard listing previously generated summaries, flashcards, quizzes and plans Each user can see only their own resources
FR-9	Role-Based Access Control	Student and Admin roles Admin-only APIs blocked for students (403 Forbidden)
FR-10	Admin Management	View and manage registered users Monitor AI usage and review system logs

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

FR No.	Non-Functional Requirement	Description
NFR-1	Usability	Simple React dashboard with a clear flow: register, log in, upload material, choose an AI feature. APIs return consistent JSON responses and clear error messages.

NFR-2	Security	JWT authentication, bcryptjs password hashing, role-based access control and input validation. The Gemini API key is kept on the server in environment variables, and users can access only their own data. HTTPS is recommended for deployment.
NFR-3	Reliability	Gemini failures or timeouts return a clear error and nothing partial is saved. Mongoose schema validation protects data quality, and generated resources are stored so they are not lost.
NFR-4	Performance	Non-blocking Node.js request handling. AI response time depends on the Gemini API, so saved results are reused instead of regenerated. Database indexes on userId and materialId and rate limiting on AI routes are planned.
NFR-5	Availability	The stateless JWT-based API can run as multiple instances behind a load balancer, and a hosted MongoDB with replication is planned for deployment. Non-AI features keep working if the Gemini API is unavailable.
NFR-6	Scalability	Modular MVC design separates routes, controllers, AI service and models. The API scales horizontally, MongoDB can be sharded as data grows, and new AI features can be added as new service modules.

References:

Node.js Documentation: <https://nodejs.org/en/docs>

Express.js Routing Guide: <https://expressjs.com/en/guide/routing.html>

MongoDB Documentation: <https://www.mongodb.com/docs/>

Mongoose ODM Documentation: <https://mongoosejs.com/docs/>

JSON Web Token Introduction: <https://jwt.io/introduction>

Google Gemini API Documentation: <https://ai.google.dev/gemini-api/docs>

React Documentation: <https://react.dev/>

OWASP Top Ten (security requirements): <https://owasp.org/www-project-top-ten/>